

Le Control Plane Kubernetes

Anatomie d'une requête, authentification, RBAC et CRDs

Cours d'architecture logicielle & déploiement — M2 Ingénierie Web

2025–2026

Table des matières

Avant-propos	2
Vue d'ensemble du Control Plane	2
Anatomie d'une requête kubectl	3
Étape 1 — Côté client (kubectl)	3
Étape 2 — Côté serveur (kube-apiserver)	4
Authentification : <i>qui es-tu ?</i>	4
Exemple concret — un certificat client X.509	5
Et pour les pods ? Les ServiceAccount	5
mTLS — l'authentification <i>mutuelle</i> par certificats	6
TLS classique vs mTLS	6
mTLS dans Kubernetes — c'est partout	7
Lien avec RBAC : CN = identité, O = groupes	8
mTLS <i>entre vos pods</i> (le sujet de l'étape 11 du TP)	8
Service Mesh — le mTLS automatique	9
Synthèse mTLS	9
Autorisation : <i>as-tu le droit ?</i>	10
Le triplet RBAC	10
Une permission, c'est : apiGroup + resource + verb	10
Construire un Role	12
La liaison : RoleBinding	12
Role vs ClusterRole	12
Exemple comparatif	13
Les ClusterRole fournis par Kubernetes	13
Admission Control : <i>cette ressource est-elle conforme ?</i>	14
Les CRDs : étendre le modèle	15
Exemple : Prometheus Operator	15
Conséquence pour le RBAC	15
L'outil de diagnostic : kubectl auth can-i	15
Synthèse — les 3 étapes de tout RBAC	16
Exercice fil rouge	17
Énoncé	17
Travail à faire	17
Vérification	17
Solution	17
À retenir (1 page)	17

Avant-propos

Ce cours répond à une seule question : « *Que se passe-t-il, exactement, entre le moment où vous tapez `kubectl apply -f pod.yaml` et le moment où votre Pod tourne dans le cluster ?* ».

Cette question paraît anodine. Elle est en réalité le pilier de tout ce que vous ferez en exploitation Kubernetes :

- les erreurs 401 Unauthorized et 403 Forbidden que vous croiserez en production ne sont pas des bugs — elles viennent de ce flux ;
- les ServiceAccounts qu'on vous demandera de configurer pour Prometheus, Argo, Velero... sont des étapes de ce flux ;
- les ClusterRole et RoleBinding que vous écrirez sont des paramètres de ce flux ;
- les Custom Resource Definitions que vous installerez (cert-manager, Istio, ...) étendent ce flux.

Comprendre le control plane, c'est donc comprendre l'API de Kubernetes. Tout le reste en découle.

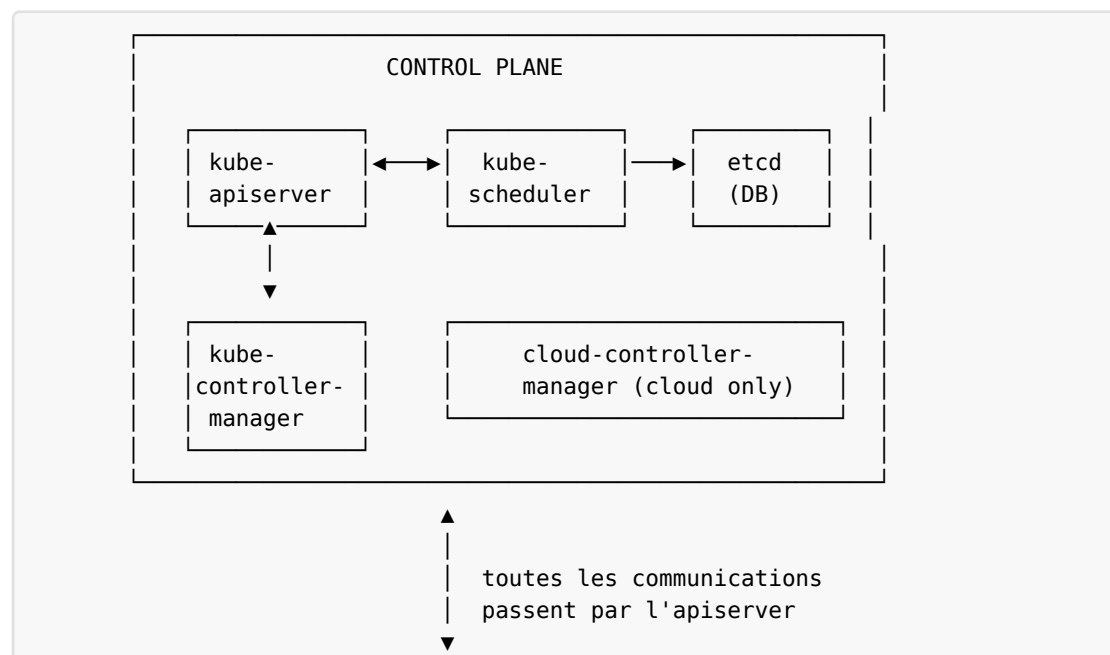
À l'issue de ce cours, vous saurez :

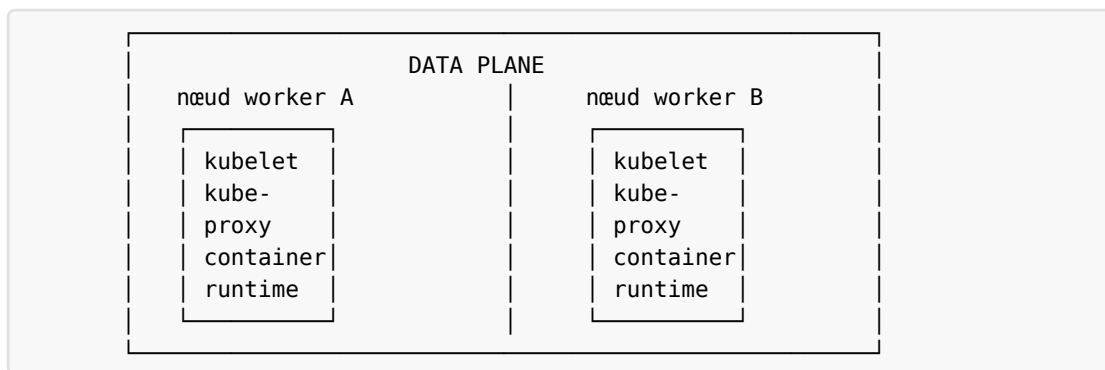
1. décrire l'anatomie d'une requête depuis `kubectl` jusqu'à `etcd`, en nommant chaque étape ;
2. expliquer pourquoi K8s ne stocke pas d'utilisateurs et comment l'authentification fonctionne quand même ;
3. concevoir un modèle RBAC propre (identité, rôle, liaison) ;
4. utiliser `kubectl auth can-i` pour auditer des permissions ;
5. distinguer un Role d'un ClusterRole, et savoir quand utiliser lequel ;
6. comprendre ce qu'est une CRD et comment elle s'insère dans le modèle.

Vue d'ensemble du Control Plane

Un cluster Kubernetes se divise en deux : le control plane (le cerveau) et les data plane (les nœuds qui exécutent vos charges).

Le control plane est lui-même composé de plusieurs processus :





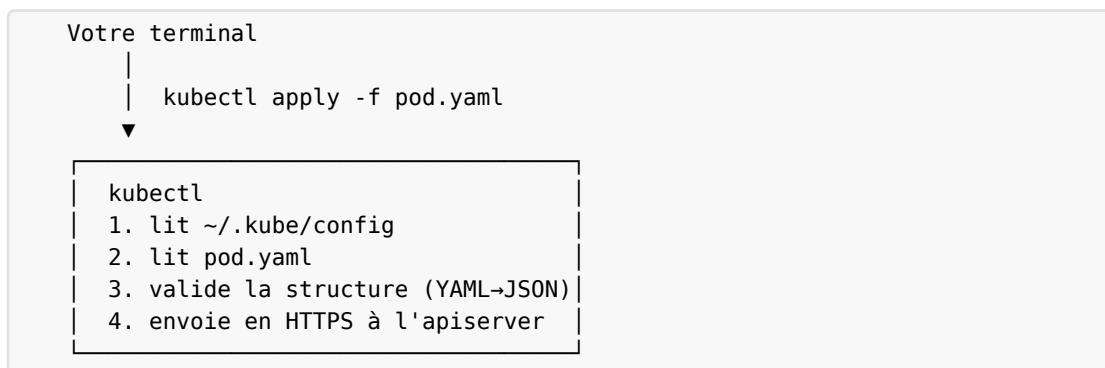
Composant	Rôle
kube-apiserver	Le point d'entrée unique du cluster. Toute commande, tout contrôleur, tout pod parle à l'apiserver — jamais directement à etcd.
etcd	Base de données distribuée (clé-valeur). Stocke toute la vérité du cluster : ressources, état, secrets.
kube-scheduler	Décide sur quel nœud va tourner chaque Pod, en fonction des contraintes (ressources, affinités, tolerations...).
kube-controller-manager	Lance les boucles de contrôle (Deployment, ReplicaSet, Node, Service...) qui rapprochent l'état réel de l'état souhaité.
cloud-controller-manager	Pareil, mais pour les ressources cloud (LoadBalancer AWS, disque GCP...). Absent en local.

Mémo : il n'y a qu'une seule porte d'entrée — kube-apiserver. C'est lui qu'on protège, c'est lui qu'on audite, c'est lui qu'on extend. Tout commence et se termine à l'apiserver.

Anatomie d'une requête `kubectl`

Commençons par le cas le plus simple : `kubectl apply -f pod.yaml`. Voici ce qui se passe.

Étape 1 — Côté client (`kubectl`)



`kubectl` n'est pas Kubernetes. C'est un simple client HTTPS qui :

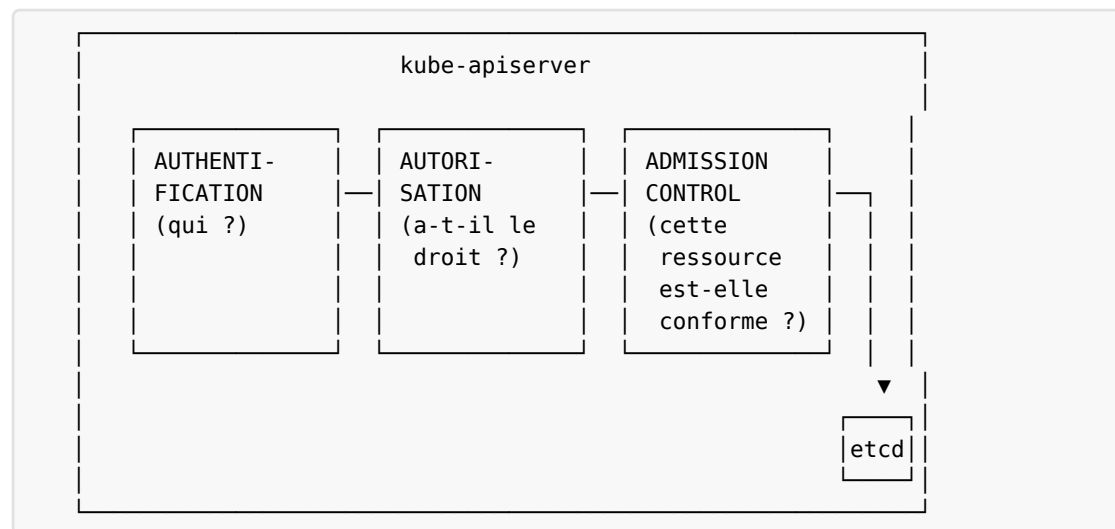
1. lit le fichier `~/.kube/config` pour savoir où parler (URL de l'apiserver) et comment s'authentifier (certificat, token, oidc...) ;
2. parse votre YAML et le convertit en JSON ;

3. fait une requête HTTPS POST (ou PUT / PATCH selon le verbe) vers l'apiserver.

Si vous voulez voir la requête exacte, ajoutez -v=8 à n'importe quelle commande kubectl. C'est très instructif.

Étape 2 — Côté serveur (kube-apiserver)

L'apiserver fait passer votre requête par trois portes successives :



Porte	Question	Échec
Authentification	<i>Qui es-tu ?</i>	401 Unauthorized
Autorisation	<i>As-tu le droit de faire cette action ?</i>	403 Forbidden
Admission Control	<i>Ce que tu crées est-il conforme aux politiques du cluster ?</i>	400 Bad Request (avec un message d'erreur explicite)

Si les trois portes répondent OK, l'objet est sérialisé puis écrit dans etcd. À partir de là, il est dit « *observable* » : les contrôleurs du cluster vont le voir et agir dessus (planification, création de Pods, etc.).

Authentification : *qui es-tu ?*

C'est ici que beaucoup d'étudiants se trompent : Kubernetes n'a pas d'objet User. Il n'y a pas de table d'utilisateurs en base. Pas de mot de passe à gérer.

Idée clé. : Kubernetes ne crée pas d'utilisateurs. Il fait confiance à un mécanisme externe pour vous identifier, et il regarde l'identité que vous présentez.

Concrètement, quand vous parlez à l'apiserver, vous présentez une preuve d'identité parmi :

Méthode	Qui l'utilise	Comment
Certificat client X.509	Humains, opérateurs, clusters managés	Le certificat est signé par la CA du cluster ; le CN devient votre nom d'utilisateur, les O deviennent vos groupes
Bearer token	ServiceAccounts (pods), CI/CD	Token JWT envoyé dans l'en-tête <code>Authorization: Bearer ...</code>
OIDC	Entreprise (Google Workspace, Azure AD, Okta...)	Token JWT signé par votre IdP, vérifié par l'apiserver
Webhook	Cas exotiques	L'apiserver appelle un service externe qui valide

Exemple concret — un certificat client X.509

Quand vous faites `kubectl apply` depuis votre machine, votre `kubeconfig` contient un certificat. Voici ce qu'il ressemble réellement :

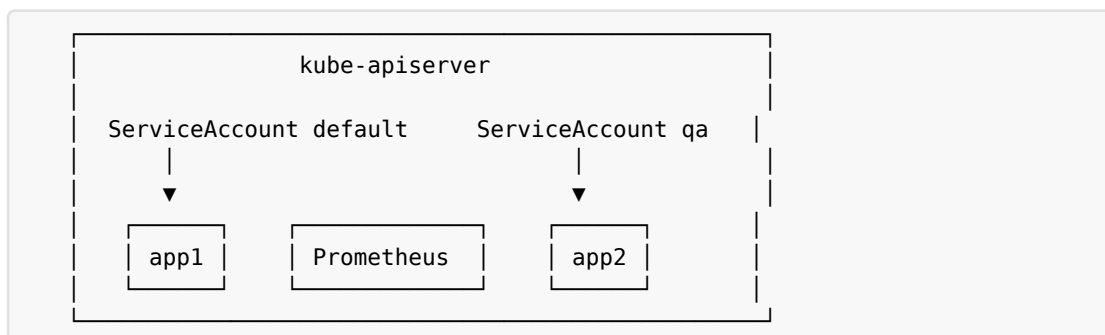
```
Certificate:
Data:
  Issuer: C=US, ST=CA, L=Los Banos, CN=DevOps by Example
  Subject: C=US, ST=CA, L=Los Banos, CN=kamel@esgi.com
  Validity:
    Not Before: Apr 7 2024
    Not After : Apr 7 2025
  Public Key Algorithm: id-ecPublicKey
  X509v3 Extensions:
    X509v3 Key Usage:          Digital Signature, Key Encipherment
    X509v3 Extended Key Usage: TLS Web Client Authentication
```

Pour Kubernetes, votre identité est `kamel@esgi.com` (le CN), parce que la CA `DevOps by Example` est connue et de confiance. Aucun mot de passe, aucun objet User. Le simple fait de présenter ce certificat suffit.

Conséquence. : si quelqu'un vole votre certificat, il *est* vous. La rotation des certificats est donc cruciale (durée de vie courte + renouvellement automatique).

Et pour les pods ? Les ServiceAccount

Un pod qui veut parler à l'apiserver (par exemple : Prometheus qui veut lister les pods pour les scraper) ne présente pas un certificat — il présente un token. Ce token est issu d'un ServiceAccount.



Quand vous ne précisez pas de `serviceAccountName` dans un Pod, K8s lui colle automatiquement le default du namespace. Le default ne devrait avoir aucune permission au-delà du minimum. Beaucoup de clusters fournissent par erreur un default trop puissant.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp
  namespace: dev
```

Une fois ce ServiceAccount créé, vous pouvez l'attacher à un Pod :

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp
  namespace: dev
spec:
  serviceAccountName: myapp
  containers:
  - name: myapp
    image: ...
```

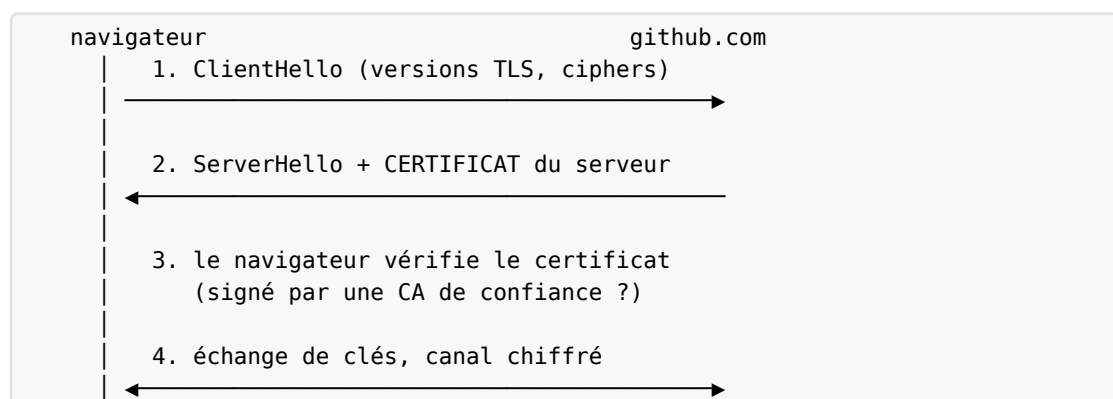
Le pod montera automatiquement, dans `/var/run/secrets/kubernetes.io/serviceaccount/`, le token JWT correspondant. C'est ce token qu'il présentera quand il appellera l'apiserver.

mTLS — l'authentification *mutuelle* par certificats

Idee clé. : si vous avez compris l'authentification par certificat client X.509 ci-dessus, vous avez à 90 % compris le mTLS. Ajoutez juste un mot : *mutuelle*.

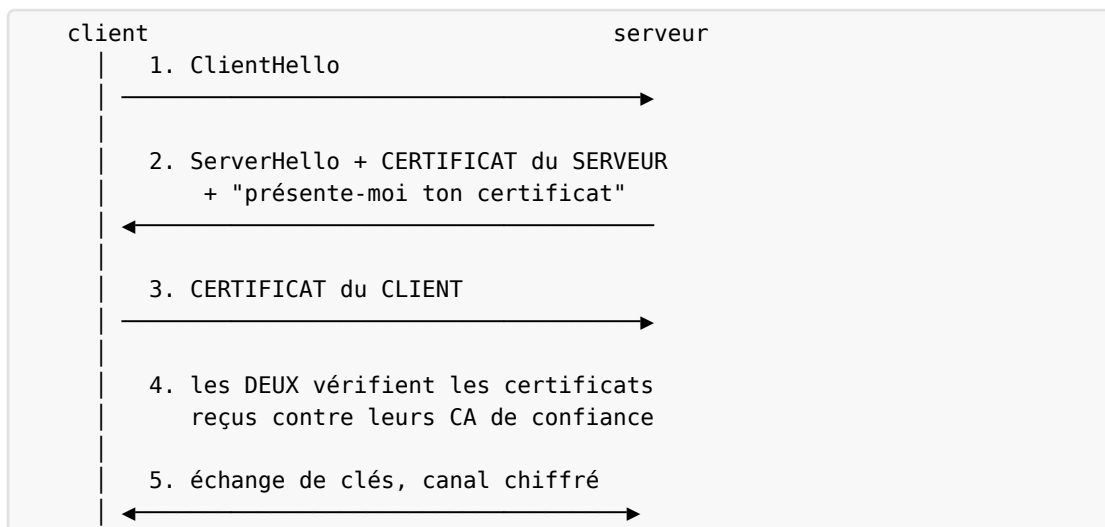
TLS classique vs mTLS

Quand vous ouvrez <https://github.com> dans un navigateur, il se passe ceci :



Un seul certificat est présenté : celui du serveur. Le client (navigateur) reste anonyme. Le serveur ne sait pas *qui* tu es, juste que toi tu sais que c'est bien lui. C'est asymétrique.

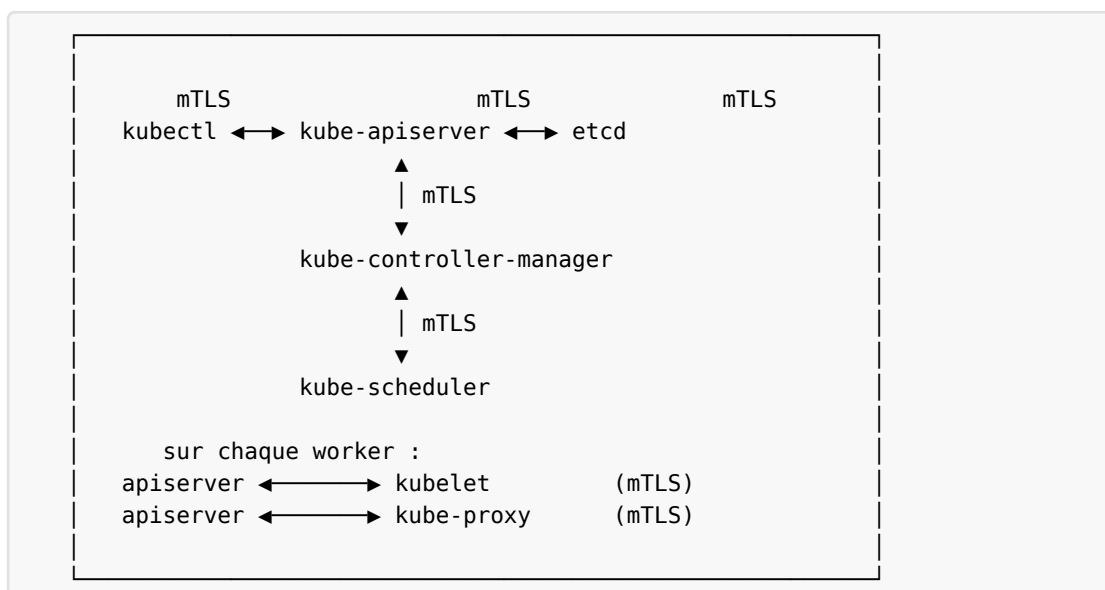
En mTLS, on ajoute une étape :



Désormais, le serveur connaît l'identité du client (via le CN du certificat client) et peut décider — RBAC, journalisation, contrôle d'accès. Le client reste sûr de parler au bon serveur. C'est symétrique.

mTLS dans Kubernetes — c'est partout

Vous croyez que le mTLS est une feature exotique ? Kubernetes ne tourne pas sans. Tout le control plane fonctionne en mTLS :



Quand kind ou kubeadm provisionne un cluster, il génère silencieusement dix certificats : un par composant (apiserver, etcd-server, etcd-peer, controller-manager, scheduler, kubelet, front-proxy...), tous signés par une CA interne du cluster (/etc/kubernetes/pki/ca.crt).

À voir une fois dans sa vie : sur un cluster kubeadm, faites `ls /etc/kubernetes/pki/`. Vous y trouverez la CA et les certs de chaque composant. C'est votre cluster que vous regardez à nu.

Lien avec RBAC : cn = identité, o = groupes

Quand un client mTLS contacte l'apiserver :

```
Subject du certificat client :  
  CN = kamel@esgi.com  
  O = developers  
  O = staging-readers
```

L'apiserver traduit :

- CN (Common Name) → nom d'utilisateur au sens RBAC ;
- O (Organization) → groupe(s) au sens RBAC.

Donc les RoleBinding peuvent référencer le user kamel@esgi.com ou le groupe developers, sans qu'aucun objet User ou Group n'existe en base. C'est le certificat lui-même qui est l'identité.

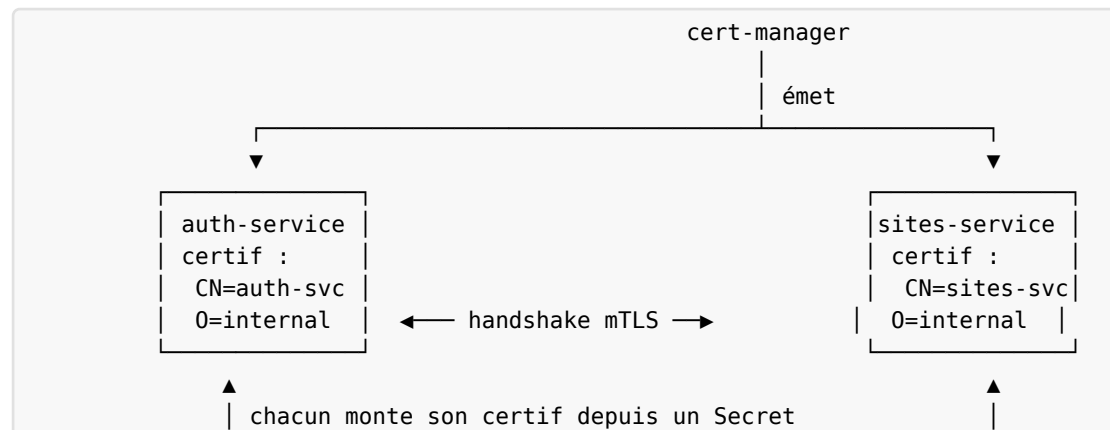
```
apiVersion: rbac.authorization.k8s.io/v1  
kind: RoleBinding  
metadata:  
  name: dev-team-binding  
  namespace: staging  
roleRef:  
  kind: ClusterRole  
  name: edit  
  apiGroup: rbac.authorization.k8s.io  
subjects:  
- kind: Group  
  name: developers      # ← matche le `O` du certificat  
  apiGroup: rbac.authorization.k8s.io
```

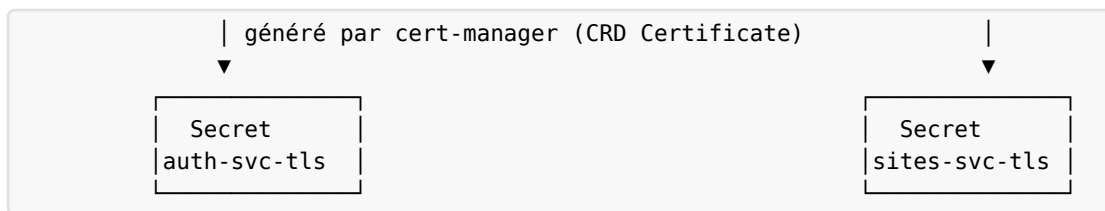
À retenir : créer un certificat avec O=developers et l'envoyer à un dev *suffit* à lui donner les droits — pas besoin d'inscrire le dev dans une base d'utilisateurs.

mTLS entre vos pods (le sujet de l'étape 11 du TP)

Le control plane est sécurisé en mTLS d'origine. Mais le trafic entre vos propres pods, par défaut, ne l'est pas. Un Pod A peut joindre un Pod B en HTTP simple, sans aucune authentification.

C'est exactement ce que vous corrigez à l'étape 11 du TP :





Trois ingrédients :

1. Une CA interne (le Certificate `salleenfrance-ca` dans `cert-manager`) qui signe tous les certs applicatifs.
2. Un certificat par service, émis par cette CA, avec le bon CN et les bons `dnsNames` (les noms DNS K8s du service).
3. Côté code applicatif : le serveur charge `tls.crt`, `tls.key`, et la `ca.crt`, puis configure `requestCert: true`, `rejectUnauthorized: true`. Côté client, on charge la même `ca.crt` plus son propre cert client.

Piège classique. : oublier que la sonde K8s `livenessProbe` n'a pas de certificat client. Si vous activez le mTLS sur le port applicatif, la probe est rejetée → le pod paraît `Unhealthy` → redémarrage en boucle. Solution : exposer un second port HTTP simple (8080) qui sert uniquement `/healthz` aux probes, sans mTLS. C'est ce qui est demandé dans le TP.

Service Mesh — le mTLS automatique

Configurer le mTLS à la main dans chaque service (comme on le fait dans le TP) est formateur mais pénible. En production, on installe un service mesh :

Mesh	Comment il fait le mTLS
Linkerd	Injecte un <i>sidecar</i> <code>linkerd-proxy</code> à côté de chaque Pod. Tout le trafic sortant passe par le proxy qui ajoute le mTLS. Certs gérés par <code>linkerd identity</code> , rotation toutes les 24h.
Istio (mode <i>ambient</i>)	Idem, mais avec un proxy partagé par nœud (<code>ztunnel</code>) au lieu d'un sidecar par pod. Plus léger.
Cilium	Met le mTLS au niveau de la pile réseau eBPF, sans sidecar du tout.

Dans tous les cas, le développeur applicatif n'écrit pas une ligne de TLS. Le mesh s'en charge.

Alors pourquoi vous le faire à la main dans le TP ? Parce qu'avant de déléguer une chose, il faut l'avoir comprise. Le jour où votre cluster bascule en Linkerd, vous saurez ce qui se passe sous le capot — et vous saurez déboguer.

Synthèse mTLS

1. mTLS = TLS où les deux parties s'authentifient par certificat.
2. Tout Kubernetes tourne déjà en mTLS entre ses propres composants. Vous bénéficiez de cette sécurité gratuitement.
3. Pour vos pods, mTLS n'est pas activé par défaut — vous l'ajoutez via `cert-manager` ou via un service mesh.
4. Identité = certificat. Pas de table `users` en base. CN → user, 0 → groups.

5. Probes K8s + mTLS : prévoir un port HTTP simple dédié.
6. Rotation : certificats courts (24h) avec renouvellement auto (renewBefore) ; cert-manager s'en occupe.

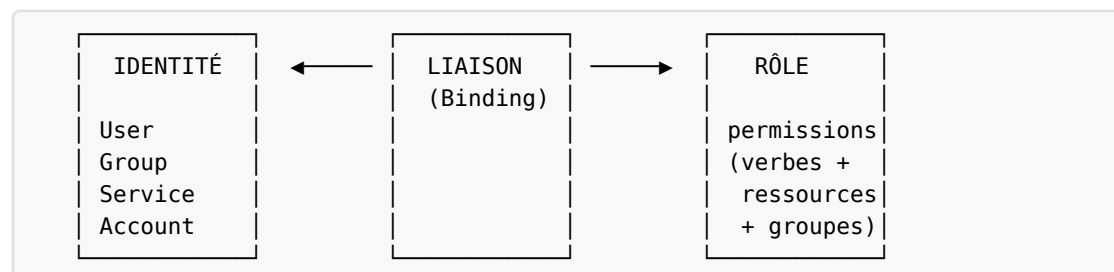
Autorisation : *as-tu le droit ?*

Une fois authentifié, vous êtes une identité (humain ou ServiceAccount). Mais ça ne suffit pas — il faut encore que vous ayez le droit de faire ce que vous demandez.

C'est le rôle du RBAC (*Role-Based Access Control*).

Le triplet RBAC

Idée clé. : RBAC repose sur trois types d'objets que vous devez systématiquement penser ensemble.



1. Identité — qui ? (un User, un Group, ou un ServiceAccount)
2. Rôle — quoi ? (une liste de permissions : « *GET sur les pods, LIST sur les services...* »)
3. Liaison — *cette identité a ce rôle*. Sans la liaison, identité et rôle s'ignorent.

C'est un schéma classique en sécurité : on ne donne jamais de droits directement à un utilisateur ; on donne des droits à un rôle, et on attache l'utilisateur au rôle. Un changement de poste = on rebascule la liaison, le rôle ne bouge pas.

Une permission, c'est : **apiGroup + resource + verb**

Une permission est toujours la combinaison de trois éléments :

Champ	Exemple	Signification
apiGroups	" " (core) ou apps ou monitoring.coreos.com	À quel groupe d'API appartient la ressource ?
resources	pods, services, prometheuses	De quelle ressource parle-t-on ?
verbs	get, list, watch, create, ...	Quelle action ?

Les verbes

Verbe	Sens
get	Lire une ressource (par son nom)
list	Lister toutes les ressources d'un type dans un scope
watch	S'abonner aux changements (stream temps réel)
create	Créer une nouvelle ressource
update	Remplacer entièrement une ressource existante
patch	Modifier partiellement une ressource
delete	Supprimer une ressource
deletecollection	Supprimer plusieurs ressources d'un coup
impersonate	Se faire passer pour un autre utilisateur (rare, admin)
escalate	Augmenter ses propres privilèges (très rare)
bind	Lier un rôle à une identité

Les groupes d'API et les ressources principales

Catégorie	apiGroups	Ressources
Core	" "	Pods, services, endpoints, configmaps, secrets, namespaces, nodes, persistentvolumes, persistentvolumeclaims, serviceaccounts
Apps	apps	deployments, daemonsets, replicaset, statefulsets
Batch	batch	jobs, cronjobs
Networking	networking.k8s.io	networkpolicies, ingresses
RBAC	rbac.authorization.k8s.io	roles, clusterroles, rolebindings, clusterrolebindings
Autoscaling	autoscaling	horizontalpodautoscalers
Storage	storage.k8s.io	storageclasses, volumeattachments
Custom	apiextensions.k8s.io	customresourcedefinitions

Attention — le piège de apiGroups: [""]. Le tableau vide signifie « *le groupe core* ». Il n'inclut pas les ressources des autres groupes. Si vous écrivez :

```
rules:  
- apiGroups: [""]
```

```
resources: ["deployments"]
verbs: ["get"]
```

ça ne fonctionnera pas — les deployments sont dans le groupe apps, pas core. La règle est correcte mais vise une ressource qui n'existe pas dans le groupe ciblé. Pour des Deployment, il faut écrire apiGroups: ["apps"].

Construire un Role

Un Role regroupe une ou plusieurs règles. Une règle = (apiGroups, resources, verbs).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: viewer
  namespace: dev
rules:
  # Règle 1 : lecture seule des pods et services du core
  - apiGroups: [""]
    resources: ["services", "pods"]
    verbs: ["get", "list"]

  # Règle 2 : lister les CRDs (groupe apiextensions)
  - apiGroups: ["apiextensions.k8s.io"]
    resources: ["customresourcedefinitions"]
    verbs: ["list"]

  # Règle 3 : lecture des Prometheus (CRD)
  - apiGroups: ["monitoring.coreos.com"]
    resources: ["prometheuses", "prometheuses/status"]
    verbs: ["get"]
```

Ce Role ne fait rien tant qu'il n'est pas lié à une identité.

La liaison : RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: myapp-viewer
  namespace: dev
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: viewer          # ← le rôle ci-dessus
subjects:
  - kind: ServiceAccount
    name: myapp          # ← l'identité
    namespace: dev
```

Maintenant, le ServiceAccount myapp (et donc tout pod qui l'utilise) peut faire get et list sur les pods et services de dev. C'est tout. Pas plus. Pas dans prod. Pas sur les secrets.

Role vs ClusterRole

C'est le point qui revient le plus en interview Kubernetes. Mémorisez le tableau.

	Role	ClusterRole
Portée	un seul namespace	tout le cluster
Définit des permissions sur	ressources <i>namespaced</i> (pods, services, configmaps...)	ressources <i>globales</i> (nodes, persistentvolumes, namespaces...) et aussi ressources <i>namespaced</i>
Liaison associée	RoleBinding	ClusterRoleBinding (ou RoleBinding pour limiter à un namespace)
Cas d'usage typique	donner des droits à un dev dans staging	donner des droits globaux (admin, lecture seule, observateur cluster)
Exemples de ressources	pods, services, secrets, deployments, jobs	nodes, persistentvolumes, namespaces, clusterroles, crds

Règle pratique : si la ressource visée est *globale* (n'a pas de namespace dans son YAML), il faut un ClusterRole. Un Role ne peut pas accorder de permissions sur des ressources globales — même si vous les listez dans rules, ça sera silencieusement ignoré.

Exemple comparatif

```
# Role – limité au namespace, mais erreur logique : nodes/PV sont globaux
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: viewer
  namespace: default
rules:
- apiGroups: [""]
  resources: ["persistentvolumes", "nodes"] # ← invalide !
  verbs: ["get", "list", "watch"]
```

```
# ClusterRole – équivalent CORRECT
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: viewer
rules:
- apiGroups: [""]
  resources: ["persistentvolumes", "nodes"]
  verbs: ["get", "list", "watch"]
```

Les ClusterRole fournis par Kubernetes

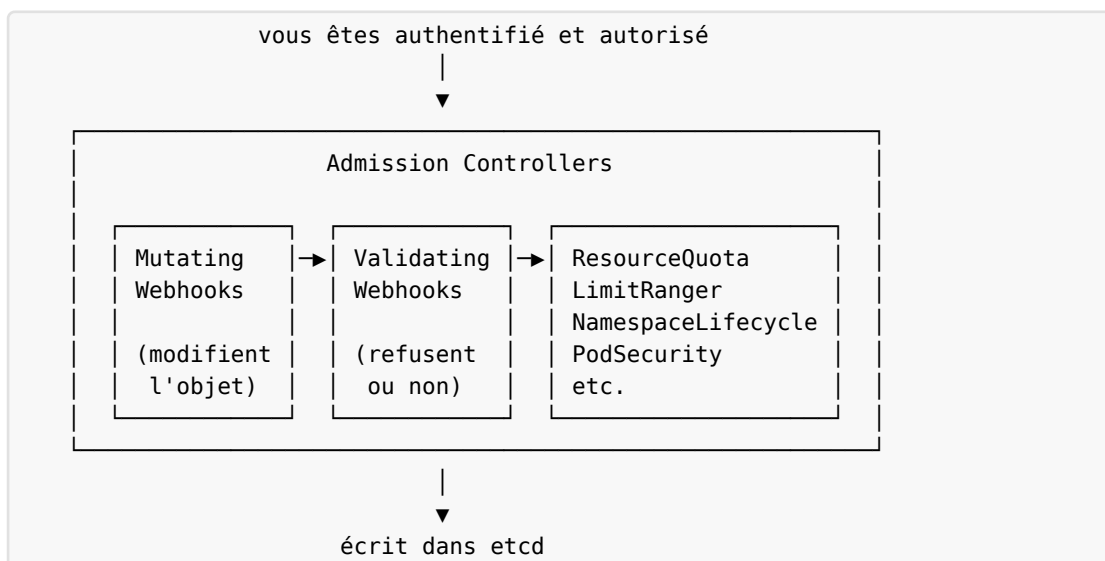
Kubernetes livre déjà un certain nombre de ClusterRole prêts à l'emploi. Vous n'avez pas à réinventer la roue.

ClusterRole	Description
cluster-admin	Accès complet au cluster (à manipuler avec une infinie prudence)
admin	Accès complet dans un namespace (via un RoleBinding)
edit	Peut modifier la plupart des objets dans un namespace
view	Lecture seule dans un namespace
system:node	Rôle du kubelet sur chaque nœud
system:controller:*	Rôles internes des contrôleurs
system:auth-delegator	Permet d'utiliser l'authentification déléguée
system:discovery	Permet la découverte des API (<code>kubectl get --raw /</code>)

Bonne pratique : pour un dev, un RoleBinding qui lie view ou edit à son user dans staging vaut souvent mieux qu'un Role custom qu'il faudra maintenir.

Admission Control : *cette ressource est-elle conforme ?*

C'est la troisième porte. Une fois que vous êtes authentifié et autorisé, l'apiserver soumet l'objet à une chaîne de contrôleurs d'admission.



Quelques exemples concrets :

- PodSecurity refuse un Pod qui demanderait `privileged: true` dans un namespace marqué `restricted` ;
- ResourceQuota refuse un Pod qui ferait dépasser le quota du namespace ;
- LimitRanger *injecte* des requests par défaut si vous n'en mettez pas (mutating) ;
- cert-manager (que vous installez dans le TP) ajoute un *Mutating Webhook* qui détecte les Ingress avec une annotation et leur greffe automatiquement un certificat TLS.

Si une `kubectl apply` est refusée avec un message du type « *admission webhook X denied the request: ...* », vous savez exactement où regarder.

Les CRDs : étendre le modèle

Jusqu'ici, on a parlé des ressources *standards* : pods, services, deployments, etc. Mais Kubernetes laisse aussi à n'importe qui le droit d'ajouter ses propres ressources.

Cette extension passe par une CustomResourceDefinition (CRD).

Exemple : Prometheus Operator

L'opérateur Prometheus installe une CRD Prometheus :

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: prometheuses.monitoring.coreos.com
spec:
  group: monitoring.coreos.com
  names:
    kind: Prometheus
    listKind: PrometheusList
    plural: prometheuses
    singular: prometheus
    shortNames: [prom]
  scope: Namespaced
```

Une fois cette CRD installée, vous pouvez créer des objets Prometheus comme s'ils étaient natifs :

```
$ kubectl get prometheus -n dev
```

NAME	VERSION	DESIRED	READY	RECONCILED	AVAILABLE	AGE
main	v2.50.0	1	1	True	True	6m5s

L'apiserver ne sait pas quoi faire avec cet objet — il sait juste le stocker. C'est un opérateur (un contrôleur custom) qui va le voir et matérialiser le Prometheus en pods, services, etc.

Toutes les briques que vous installerez en TP sont des CRDs : cert-manager ajoute Certificate, Issuer, ClusterIssuer ; un service mesh ajoute VirtualService, Gateway... Le modèle est le même.

Conséquence pour le RBAC

Une CRD a son propre apiGroup. Pour donner accès à prometheuses, on n'écrit pas apiGroups : [""] (cela viserait les ressources core). On écrit :

```
rules:
- apiGroups: ["monitoring.coreos.com"]
  resources: ["prometheuses", "prometheuses/status"]
  verbs: ["get"]
```

Il n'existe pas de wildcard "*" sur les groupes — ou plutôt, il faut l'écrire explicitement : apiGroups: [""]. Mais c'est très rarement la bonne réponse.

L'outil de diagnostic : kubectl auth can-i

Devant une erreur Forbidden, ne devinez pas. Demandez à l'apiserver lui-même.

```
kubectl auth can-i list nodes --as system:serviceaccount:default:my-sa
# yes / no
```

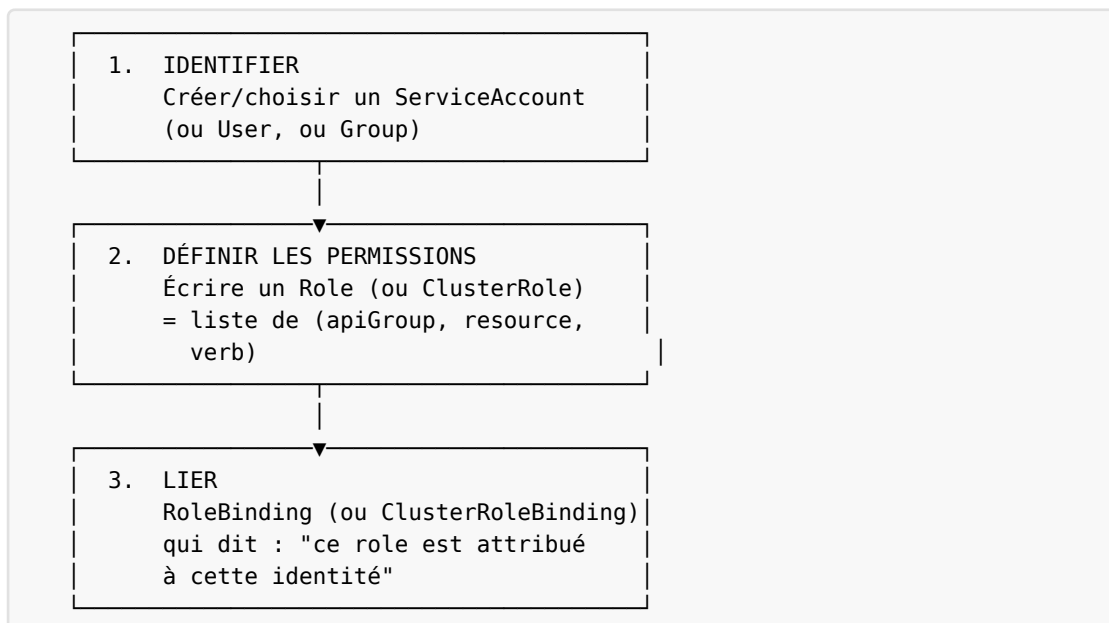
Cette commande simule l'action et renvoie la réponse exacte du sous-système RBAC, sans rien créer ni modifier.

Commande	Ce qu'elle teste
<code>kubectl auth can-i create pods</code>	Puis-je créer un pod dans mon namespace courant ?
<code>kubectl auth can-i delete pods -n dev</code>	Puis-je supprimer un pod dans le namespace dev ?
<code>kubectl auth can-i get pods --as alice</code>	Alice peut-elle lire les pods ?
<code>kubectl auth can-i --list --as system:serviceaccount:default:my-sa</code>	Quelles permissions a ce ServiceAccount, au total ?
<code>kubectl auth can-i '*' '*' --all-namespaces</code>	Suis-je cluster-admin ?

La forme `--as system:serviceaccount:<namespace>:<sa-name>` se prononce « *en tant que* ». Vous devez vous-même avoir l'impersonate pour pouvoir l'utiliser (en général, seuls les admins l'ont).

Synthèse — les 3 étapes de tout RBAC

Pour donner des droits à un pod (ou à n'importe quelle identité) :

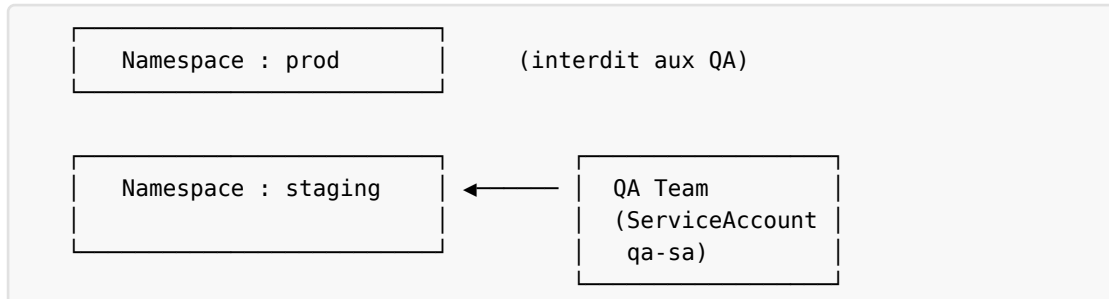


C'est cette chaîne qu'on retrouve dans 100 % des configurations RBAC. Si l'un des trois maillons manque, ça échoue à l'autorisation.

Exercice fil rouge

Énoncé

L'équipe Quality Assurance doit pouvoir tester l'application déployée dans le namespace staging, mais en aucun cas toucher à prod.



Travail à faire

Écrire les 3 ressources YAML qui réalisent cette politique :

1. un ServiceAccount qa-sa dans le namespace staging ;
2. un Role qa-tester dans staging qui autorise get / list sur services, pods et aussi pods/log (les logs sont une sous-ressource) ;
3. un RoleBinding qui lie qa-sa à qa-tester.

Vérification

Après `kubectl apply`, exécutez les commandes suivantes — chaque résultat est imposé.

```
# Doit retourner les pods de staging (yes)
kubectl get pods -n staging --as=system:serviceaccount:staging:qa-sa

# Doit échouer avec 403 Forbidden
kubectl get pods -n prod --as=system:serviceaccount:staging:qa-sa

# Doit afficher les logs d'un pod staging (oui – pods/log est dans le Role)
kubectl logs -f myapp -n staging --as=system:serviceaccount:staging:qa-sa
```

Solution

À retenir (1 page)

1. Tout passe par kube-apiserver. C'est la seule porte du cluster. etcd n'est jamais joint directement.
2. 3 portes successives dans l'apiserver : *Authentication* → *Autorisation* → *Admission Control*. Chaque porte renvoie un code HTTP différent en cas d'échec (401, 403, 400).
3. Pas d'objet User dans Kubernetes. L'identité vient de l'extérieur (certificat X.509, OIDC, token de ServiceAccount).
4. Les pods s'authentifient via leur ServiceAccount. Le default ne devrait avoir aucun droit non-essentiel.
5. RBAC = identité + rôle + liaison. Sans la liaison, le rôle est inerte.

6. Une permission est apiGroup + resource + verb. Le groupe "" est *core*, il n'inclut pas les autres.
7. Role = un namespace, ClusterRole = tout le cluster. Les ressources globales (nodes, persistentvolumes, namespaces) exigent un ClusterRole.
8. Devant un 403, ne devinez pas, lancez `kubectl auth can-i ...`.
9. Une CRD étend l'API. Elle vit dans son propre groupe d'API ; pensez-y dans vos règles RBAC.
10. Les contrôleurs d'admission (PodSecurity, LimitRanger, webhooks de cert-manager...) refusent les ressources non conformes — c'est la troisième barrière de sécurité.